

The tag reporting-rate setting is not a neutral uncertainty axis

Two investigations into the tagged-fish reporting-rate penalty. The first asked why the `RR` grid axis moves that penalty. The second asked whether one setting of the axis inflates the estimated reporting rates themselves, and whether that reaches stock status. It does both.

79 of 94 named checks held · 88 retained models · 80 with a final PAR · 168 fixed-parameter MFCL evaluations

Executive summary — in plain language

Tuna are tagged, released, and some are later recaptured and reported. The stock assessment has to assume what fraction of recaptures actually get reported — the **reporting rate**. After a tag is released there is a settling-in period, called the **mixing period**, during which the tagged fish have not yet spread through the population and the model is not supposed to take the recapture data at face value.

MFCL offers two ways to handle recaptures that occur during that settling-in period. The 2026 ensemble uses both, treating the choice as a legitimate source of uncertainty and splitting the models roughly evenly between them. **The two options are not equally valid.**

One option — called **exclude**, the setting that does *not* correct for reporting — quietly inserts terms into the model's fitting criterion whose best possible answer is always “reporting is 100%”. Those terms are not information about the fishery. They are an artefact of the setting. They push the estimated reporting rates upward, away from what the independent tag-seeding studies say they should be.

This matters because reporting rates and fishing mortality trade off against each other. If the model believes more of the recaptures were reported, it needs less fishing to explain the tags that came back. Models using **exclude** therefore estimate lower fishing mortality and a healthier stock — and the difference is not small:

CHANCE FISHING EXCEEDS THE LIMIT 21% vs 39% exclude models vs include models (reported ensemble: 30%)	CHANCE STOCK IS BELOW THE LIMIT 17% vs 32% exclude vs include (reported ensemble: 24%)	FISHING MORTALITY -18% F/F_{MSY} under exclude, adjusted for every other axis
--	---	---

So the probability statements that go to the Scientific Committee shift by roughly half their own value depending on a modelling convention, not on data or biology.

The other option is better, but not clean. The **include** setting does not create the artefact — we confirmed this both by reading the MFCL source code and by a controlled numerical experiment. But it is numerically fragile: it divides the observed recaptures by the estimated reporting rate, which can produce impossibly large numbers,

and MFCL then applies an internal safety floor. That floor engaged in **every single include** model, and where it engages the same artefact partially returns. **Include** models also failed to converge three times as often (18% vs 6%).

A separate defect, worth reporting on its own. The tag-fit diagnostic plots circulated with assessments cannot detect problems in the mixing period. In that period the plotted “predicted” recaptures are forced to equal the observed ones by construction, in both settings — so a good-looking fit there is arithmetic, not evidence. Under **exclude** the numbers the model actually fitted differ from the numbers plotted by about 21.6%.

What we recommend

Include is the more defensible of the two settings on statistical grounds, and the ensemble should not present the choice as a balanced uncertainty axis. But the case is not free: **include** carries a real numerical-robustness cost, and the MFCL manual itself recommends the opposite setting for exactly that reason. The honest position is that this is a trade-off between a *known statistical contamination* and a *known numerical fragility*, and it has been treated as neither.

The rest of this document sets out the evidence. Every directional claim names a check that either held or did not, and failures are reported in the same form as passes.

PART 1 · THE ORIGINAL QUESTION

Why the reporting-rate penalty moves with the axis

The starting observation was that among all the objective-function components, the **tagged-fish reporting-rate penalty** is the one the **RR** axis really moves — a median near 60 under **include** against 100 under **exclude**.

That penalty is a prior. Three tag-seeding studies give expected reporting rates for the purse-seine fisheries in regions 2, 3-west and 3-east; the penalty charges the model for departing from them. Its form, transcribed from `multifan-cl/src/callpen.cpp`, is

$$\text{penalty} = \sum \text{ over unique reporting-rate groups: } w_g \cdot (\hat{X}_g - \text{target}_g/100)^2$$

Reconstructing it from the PAR files reproduced the reported component to 4.9×10^{-10} across all 80 members — the arithmetic is settled, and everything downstream rests on it.

HELD task3.reconstruction-matches-reported

max|resid| = 4.86e-10 (tol 1e-06); median resid = 2.32e-12

Three structural facts shape everything that follows. Only **five** reporting-rate groups carry an informative prior (not three, as originally assumed); the penalty's *level* is dominated by group 17, but the *gap between arms* comes from groups 18 and 7; and the whole effect is localised in the reporting-rate prior rather than smeared across the fit.

The obvious worry was that natural mortality was doing the work and leaking in through an unbalanced selection of models. It is not: M0 is balanced across the arms, and adjusting for it *increases* the estimated axis effect.

HELD

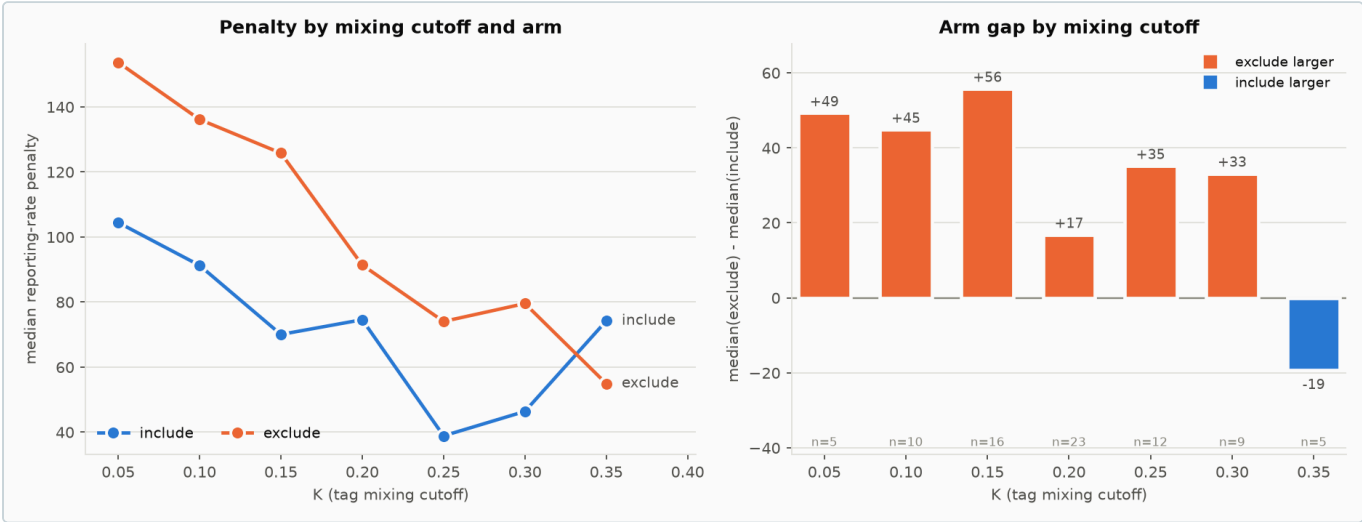
task4.M0-balanced-across-arms

standardised mean difference (exclude - include) = -0.076

HELD

task4.adjusted-arm-effect-positive

adjusted arm effect (exclude - include) = 35.54 [28.72, 42.36]



The gap tracks the mixing window: largest where windows are longest (small K) and gone where they collapse (K = 0.35). That is the first sign the mechanism lives in the mixing period.

PART 2 · THE MECHANISM

Channel 2 — how **exclude** manufactures evidence for perfect reporting

The first investigation explained the gap by a *removal* argument: the reporting rate has an extra lever under **include**, so a smaller excursion suffices. That account is correct but says nothing about *which direction* the rates move. A second mechanism does, and it is the one that matters.

In one paragraph

Recaptures during the mixing period *do* get scored by the likelihood under both settings. What differs is what the model compares them against. MFCL solves for a fishing mortality that removes a target number of tags from the released cohort, and the setting changes that target. Writing **R** for the observed returns and **$\hat{X}$** for the estimated reporting rate:

SETTING	REMOVAL TARGET	PREDICTED REPORTED RETURNS	CONSEQUENCE
include	$R / \hat{X}$	$\hat{X} \cdot (R/\hat{X}) = R$	$\hat{X}$ cancels — the terms say nothing about the reporting rate
exclude	R	$\hat{X} \cdot R$	minimised at $\hat{X} = 1$ — the terms assert perfect reporting

Under **exclude** those terms are not evidence about the tagged population. They are pseudo-data. Every mixing-period recapture becomes a small vote for “reporting is complete”, and the tag-seeding prior is the only thing pushing back.

Confirmed two ways

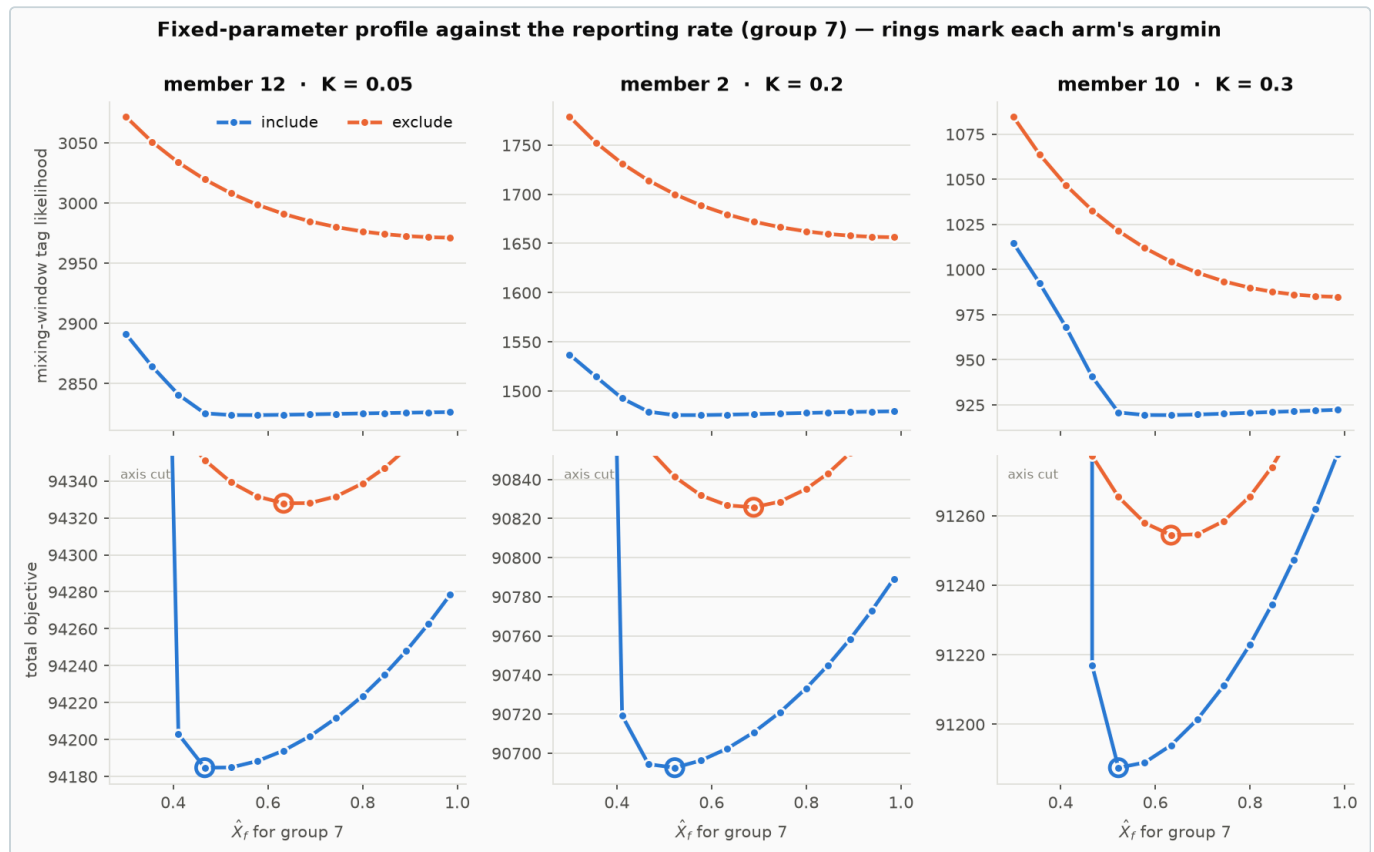
First by transcription from the production source (`src/tag3.cpp:2810` , `src/tagfit.cpp:182`) — not inferred. Second by a controlled experiment: take a converged model, freeze every parameter, and sweep one group's reporting rate under each setting. Nothing can re-estimate, so nothing can compensate.

HELD `task1.exclude-mixing-block-monotone-toward-one`

first differences of the exclude mixing block are negative in 100.0% of grid steps (worst profile 100.0%)

HELD `task1.include-mixing-block-invariant-to-X-above-0.8`

worst relative range of the include mixing block for $X \geq 0.8 = 0.0086$; median 0.0009 -- this is the sub-range where the raised removal is small enough that the survival floor should not rescale the Newton-Raphson target



The signature, at frozen parameters. Top: under **include** the mixing-window likelihood is flat in the reporting rate (0.09% median variation above $\hat{X} = 0.8$); under **exclude** it slopes toward $\hat{X} = 1$ in 100% of grid steps. Bottom: the best-fitting reporting rate sits **higher** under **exclude** in every member (+0.111 to +0.167 in $\hat{X}$ for group 7).

Where the mechanism is refined rather than confirmed

The pre-registered stop condition was that the **include** curve must be flat. Over the whole sweep it is not — up to 22.5% variation. The reason is in the source: when the raised removal $R/\hat{X}$ is too large to be feasible, MFCL applies a **pos fun** survival floor that rescales the target, and that rescaling reintroduces an $\hat{X}$ dependence of the same form as **exclude**. The floor engages at low $\hat{X}$, which is exactly where the curve bends.

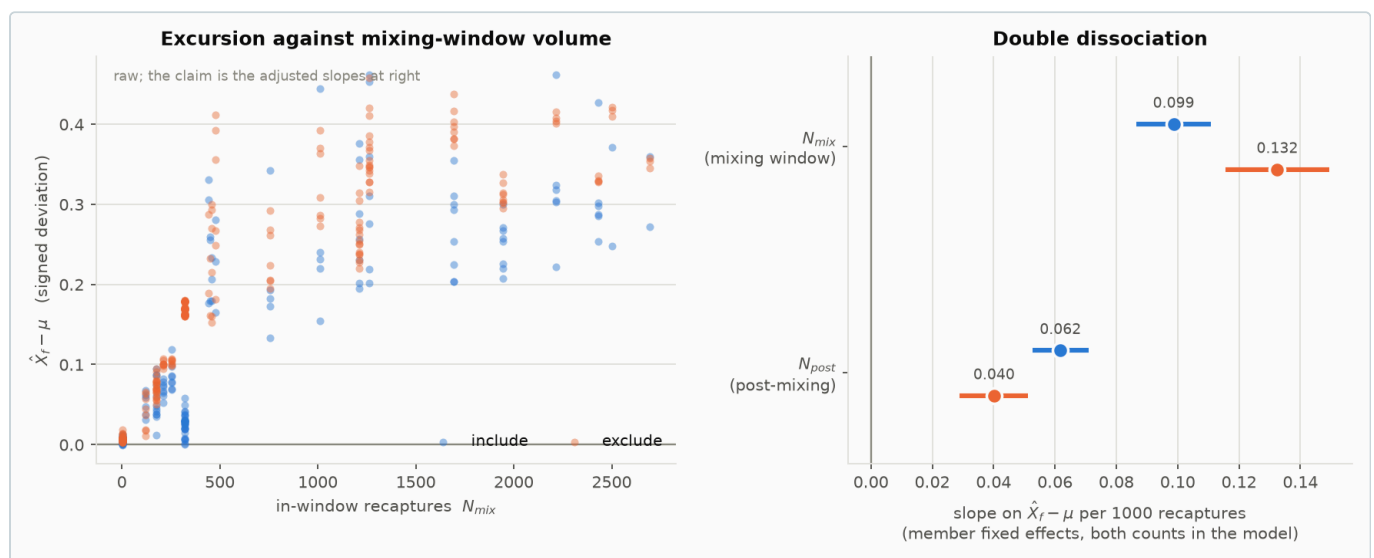
DID NOT HOLD task1.include-mixing-block-invariant-to-X

worst relative range of the include mixing block over the full grid = 0.2254 (tolerance 0.02); median 0.0701

So the correct statement is not “**include** is clean”. It is: **the artefact is pervasive under exclude and localised under include** — confined to the release groups where the floor engages, a median of 2 of 98 per model.

What orders the pull

The pull scales with the *count* of mixing-period recaptures attributed to a group, resisted by that group's prior weight — not with the in-window *share*, which is why an earlier ψ -based test found nothing. The decisive evidence is a double dissociation against post-mixing returns, which carry the same identification benefit but none of the pseudo-data pull.



Under **exclude** the mixing-window returns pull *harder* (+34%) while post-mixing returns pull *less* (–35%), both intervals excluding zero. That opposite movement is what re-weighting toward pseudo-data looks like, and it is what separates the mechanism from a generic “more data means more movement” effect.

HELD task3.Nmix-and-Npost-arm-effects-have-opposite-signs

N_{mix} slope $9.868e-05 \rightarrow 1.325e-04$ (+34%) while N_{post} slope $6.177e-05 \rightarrow 4.014e-05$ (–35%); both interaction CIs exclude zero

What it does to the advice

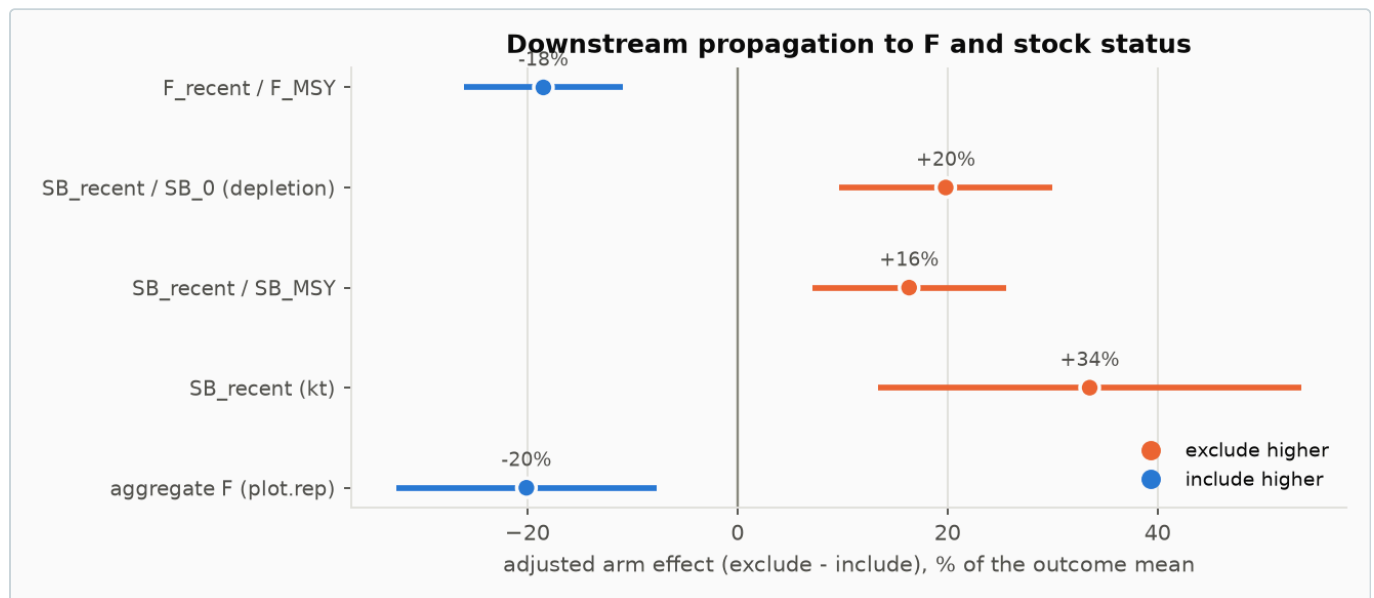
Adjusting for every other grid axis, the **exclude** setting carries **-0.166** [-0.235, -0.098] on F/F_{MSY} — about -18% of the ensemble mean — and **+0.0534** [0.0261, 0.0807] on depletion, about +20%.

HELD task5.arm-effect-on-F-is-negative[f_recent_fmsy]

F/F_{MSY} : -0.1664 [-0.2347, -0.09819], $p=5.83e-06$ (mean 0.9007)

HELD task5.arm-effect-on-depletion-is-positive

SB/SB_0 : +0.05338 [0.02608, 0.08067], $p=0.000204$ (mean depletion 0.2692)



Every management quantity moves the way the mechanism predicts, and no interval crosses zero. This is the size of the effect, not evidence for which channel caused it — both channels push F the same way.

An important negative result

We tried to decompose that effect cheaply, by measuring how much F moves per unit reporting rate with every parameter frozen. It moves **not at all**. Across 168 evaluations spanning both settings and the full range of reporting rates, the objective took 56 distinct values per model while F/F_{MSY} took **1** — a range of exactly 0.

DID NOT HOLD task9.fixed-parameter-profile-can-decompose-the-downstream-effect

it cannot: dF/dX and the removal channel are both structurally zero at frozen parameters, so the observed -18% arm effect on F/F_{MSY} is an estimation effect and only re-estimation can decompose it

The reason is structural: the reporting rate enters only the tag likelihood, its own penalty, and the tagged-cohort solver. The tagged cohort is fitted to the population and never feeds back into population numbers or fishing mortality. So the downstream effect is not arithmetic — it is **optimiser behaviour**. The reporting rate reshapes the likelihood surface and the

F-related parameters relocate during estimation.

That rules out a whole class of explanation, and it also removes the cheap instrument: only re-estimation can measure the pathway. This is why the recommendation on paired refits was reversed once the result came in.

PART 4 · THE CHOICE

Both options have real problems

EXCLUDE — no reporting-rate correction • Pseudo-data on every release group with a mixing window. The in-window terms are minimised at $\hat{X} = 1$ regardless of the data. • Estimated rates pushed up. Systematically closer to the 0.99 upper bound than include across all priored groups. • Diagnostics disagree with the fit. The objective scored in-window predictions ~21.6% below what was plotted — 802 of 3605 fish per model. • Numerically well-behaved. Only 6% of draws failed (3 of 50), and no survival-floor engagement. • It is what the MFCL manual recommends, on numerical-stability grounds.	INCLUDE — correct returns by $\hat{X}$ • No pseudo-data in the general case. The reporting rate cancels out of the in-window terms; estimates come from post-mixing returns and the prior. • But the safety floor reintroduces it locally. Engaged in 100% of models, median 2 of 98 release groups — and where it engages, the same $\hat{X} \cdot R$ form returns. • The floor penalty is discarded outright for release groups 21, 72 and 112 (hardcoded in MFCL), and group 21 is one of those that engages. • Numerically fragile. 18% of draws failed (9 of 50), three times the exclude rate — and that is informative non-response. • Goes against the manual's recommendation.
--	--

The assessment

On the specific question of whether the setting corrupts the estimated reporting rates, **the two are not symmetric and include is the more appropriate choice**. Under **exclude** the contamination is structural and universal; under **include** it is a localised consequence of a numerical guard. That asymmetry is established by source transcription and by a fixed-parameter experiment, not by inference from the ensemble.

Two honest qualifications. First, **include's** higher failure rate is *informative* non-response — the models that failed are those where the correction was least feasible, so the surviving **include** set is conditioned on the correction being benign. Second, the manual's preference for **exclude** is not wrong on its own terms; it is a numerical recommendation, and what this work adds is that following it has a statistical cost that had not been quantified.

What should change now: the ensemble should stop presenting this axis as a balanced uncertainty dimension. Whether that means reweighting, reporting both arms separately, or moving to **include**-only is a scientific judgement — but presenting a 21%-versus-39% split in limit-exceedance probability as though it were uncertainty about the stock is not defensible.

One practical point: an **include**-only ensemble already exists. The 41 retained **include** members cover every level of every other axis — all seven mixing cutoffs, all three overdispersion levels, all five effort-creep levels, and essentially the full steepness and natural-mortality ranges. Acting on the subsetting decision needs no new model runs; it needs a decision.

PART 5 · STATUS

What is established, and what is not

Established by transcription or controlled experiment

- The penalty's exact functional form, reproduced to 4.9×10^{-10} .
- That **exclude**'s mixing-period terms are minimised at $\hat{X} = 1$, and **include**'s are inert in $\hat{X}$ except where the survival floor engages.
- That flipping the flag alone, with every parameter frozen, moves the best-fitting reporting rate upward.
- That the reporting rate cannot move F at frozen parameters — the downstream effect is optimiser behaviour.
- That the in-window tag-fit diagnostic is an identity in both arms, and that under **exclude** it differs from what the objective used.

Established observationally, on a selected non-crossed ensemble

- The adjusted axis effects on the penalty (+35.5 [28.7, 42.4]), on F/F_{MSY} , and on depletion.
- The `N_mix / N_post` double dissociation.
- That **exclude** sits systematically closer to the reporting-rate bound.

Not established

- **That the downstream effect is caused by the reporting rate.** Both channels push F the same way and compound; the ensemble is observational; and conditioning on $\hat{X}$ over-mediate because $\hat{X}$ and F are jointly estimated. Only paired refits can settle this.
- **That either arm's rates are correct.** The external anchor is the tag-seeding priors, not either fit.
- **A reconciled ψ correction.** The raising scheme moves release-weighted ψ from 0.409 to 0.414 (+1.1%), consistently across all 80 models — but the mixing configuration moves it 0.563, a factor of 120 larger, and the published baseline does not reconcile on group counts.

DID NOT HOLD `task6.published-baseline-reconciles`

closest configuration is $K=0.3$: $\psi\text{-hat}$ 0.2259 vs published 0.231 (close), but 35 groups at $\psi \geq 0.3$ vs published 27 (does not reconcile)

Recommended sequence

1. **Report the diagnostic defect upstream.** The in-window tag-fit panels cannot detect misfit, and under **exclude** they disagree with the objective. This is an MFCL issue, not a configuration choice, and it needs no compute.
2. **Reconcile the ψ baseline** before any corrected figure circulates.
3. **Put the arm-split status table in front of whoever owns the advice.** A 21%-versus-39% split in limit-exceedance probability is a decision, not a footnote.
4. **Then paired refits** — now the only instrument that can measure the downstream pathway. Time one full-path run before committing to a count, and consider targeting the 9 failed **include** members, since recovering them would remove the selection concern that otherwise limits an **include-only** ensemble.

Analysis code: `analysis/rr-penalty/` and `analysis/channel2/`, reproduced by the `run-all.sh` in each. Tidy CSVs, figures and the full check ledgers in `out/` and `out/channel2/`. Source excerpts pinned to `multifan-cl` commit `624dee4f`; binary version 2.2.7.9. Objective components and the design table cover all 88 retained models; 80 have a retained `final.par`, and every quantity states its own coverage. No estimation run was launched for either investigation — every MFCL invocation set `parest_flags(1) = 0`.